



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 2

Strings

A complete guide to Java Strings — creation, the String Pool, all essential methods, comparison pitfalls, formatting, regex basics, and when to use StringBuilder.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

2.1	What is a String?	Immutable, reference type
2.2	Creating Strings & the String Pool	Literals, new, intern
2.3	Length & Character Access	length(), charAt(), toCharArray()
2.4	Comparing Strings	== vs equals vs compareTo
2.5	Searching Within Strings	indexOf, contains, startsWith
2.6	Extracting & Modifying	substring, replace, trim, case
2.7	Splitting, Joining & Conversion	split, join, valueOf, parsing
2.8	Formatting Strings	String.format, formatted
2.9	StringBuilder & StringBuffer	Efficient concatenation
2.10	Common Mistakes & Tips	Pitfalls every beginner hits

What is a String?

A `String` in Java is a **sequence of characters**. Despite looking like a primitive, it is a **fully-fledged object** from the `java.lang.String` class. It is the most used class in Java.

Key Properties

PROPERTY	DETAIL
Immutable	Once created, a String's content cannot be changed . Every "modification" creates a new String object.
Reference type	Stored on the heap. Variables hold a reference, not the characters themselves.
Final class	Cannot be subclassed — <code>class String is final</code> .
Serializable	Can be converted to a byte stream (useful for network/file I/O).
Auto-imported	In <code>java.lang</code> , so no <code>import</code> statement needed.

What Immutability Really Means

JAVA

```
1 String s = "Hello";
2 s = s + " World";
3
4 // This does NOT modify the original "Hello" string.
5 // It creates a brand new String "Hello World" and
6 // makes s point to it. The original "Hello" is unchanged.
```

IMMUTABILITY IN ACTION



NOTE

Immutability is why Strings are **thread-safe** by default and can be safely shared. It also allows Java to cache and reuse String objects (the String Pool — covered next).

Two Ways to Create a String

```
1 // Method 1: String literal (preferred)
2 String a = "Hello";
3
4 // Method 2: new keyword (creates a separate object)
5 String b = new String("Hello");
```

JAVA

Both create a valid String, but they behave differently in memory.

The String Pool (Intern Pool)

Java maintains a special memory area called the **String Pool** inside the heap. When you use a **literal**, Java checks if that value already exists in the pool:

- › **Exists?** → Reuse the existing reference
- › **Not exists?** → Create it in the pool and return the reference

STRING POOL — WHAT HAPPENS WITH EACH APPROACH

```
String a = "Hello";
```

a → "Hello" (pool)

```
String b = "Hello";
```

b → "Hello" (same!)

```
String c = new String("Hello");
```

c → "Hello" (heap, separate)

```
1 String a = "Hello";
2 String b = "Hello";
3 String c = new String("Hello");
4
5 System.out.println(a == b);    // true  (same pool object)
6 System.out.println(a == c);    // false (different objects)
7 System.out.println(a.equals(c)); // true  (same content)
```

JAVA

intern() Method

Forces a string into the pool. Returns the pooled reference if it exists, or adds it and returns the reference:

2.3 Length & Character Access

length() — Number of Characters

```
1 String s = "Hello, World!";
2 System.out.println(s.length()); // 13
```

JAVA

MISTAKE

`String.length()` has parentheses (it's a method). `Array.length` does not (it's a field). Mixing them up is a compile error:

`s.length` → compile error `arr.length()` → compile error

charAt() — Get Character at Index

```
1 String s = "Hello";
2 char c = s.charAt(0); // 'H'
3 char c2 = s.charAt(4); // 'o'
4 // char c3 = s.charAt(5); // StringIndexOutOfBoundsException!
```

JAVA

toArray() — Convert to char Array

```
1 String s = "Hello";
2 char[] chars = s.toCharArray();
3
4 for (char c : chars) {
5     System.out.print(c + " ");
6 }
7 // H e l l o
8
9 // Modify the array (the original string is unaffected)
10 chars[0] = 'J';
11 System.out.println(s); // "Hello" — unchanged
12 System.out.println(new String(chars)); // "Jello"
```

JAVA

Common Character Patterns

```
1 String s = "Hello, World!";
2
3 // Count specific character
4 int count = 0;
5 for (int i = 0; i < s.length(); i++) {
6     if (s.charAt(i) == 'l') count++;
7 }
8 System.out.println(count); // 3
9
10 // Reverse a string via char array
11 char[] chars = s.toCharArray();
12 for (int i = 0; i < chars.length / 2; i++) {
13     char temp = chars[i];
14     chars[i] = chars[chars.length - 1 - i];
15     chars[chars.length - 1 - i] = temp;
16 }
17 System.out.println(new String(chars)); // "!dlroW ,olleH"
```

isEmpty() and isBlank() (Java 11+)

```
1 String a = "";
2 String b = " ";
3 String c = "Hello";
4
5 System.out.println(a.isEmpty()); // true (length = 0)
6 System.out.println(b.isEmpty()); // false (length = 3)
7
8 System.out.println(a.isBlank()); // true
9 System.out.println(b.isBlank()); // true (only whitespace)
10 System.out.println(c.isBlank()); // false
```

METHOD	RETURNS	DESCRIPTION
length()	int	Number of characters
charAt(i)	char	Character at index <code>i</code>
toCharArray()	char[]	New char array copy
isEmpty()	boolean	<code>true</code> if length is 0
isBlank()	boolean	<code>true</code> if empty or only whitespace (Java 11+)

2.4

Comparing Strings

String comparison is one of the most common sources of bugs in Java. Understanding the difference between `=` and `.equals()` is essential.

`==` vs `equals()` — The Core Difference

```
1 String a = new String("Hello");
2 String b = new String("Hello");
3
4 System.out.println(a == b);           // false - different objects in memory
5 System.out.println(a.equals(b));      // true  - same character content
```

OPERATOR	WHAT IT CHECKS	USE FOR
<code>=</code>	Are these the same object in memory?	Almost never for Strings
<code>.equals()</code>	Do they have the same characters ?	Comparing String content (99% of cases)

WARNING

Never use `=` to compare String content. It works with literals sometimes (because of the pool), but fails with `new String()`, user input, or strings built at runtime. It is a ticking time bomb — always use `.equals()`.

`equals()` Variants

```
1 String a = "Hello";
2 String b = "hello";
3 String c = null;
4
5 System.out.println(a.equals(b));           // false (case-sensitive)
6 System.out.println(a.equalsIgnoreCase(b)); // true  (ignores case)
7
8 // Safe way to avoid NullPointerException:
9 System.out.println("Hello".equals(c));     // false (safe - "Hello" is never null)
10 // System.out.println(c.equals("Hello"));  // NullPointerException!
```

TIP

Put the **known non-null string first** when calling `.equals()`: `"known".equals(variable)`. This prevents `NullPointerException` if `variable` is `null`.

`compareTo()` — Lexicographic Comparison

Returns an `int` indicating the **lexicographic (dictionary) order**:

RETURN VALUE	MEANING
< 0	Calling string comes before the argument
0	Strings are equal
> 0	Calling string comes after the argument

```
1 System.out.println("Apple".compareTo("Banana")); // negative ('A' < 'B')
2 System.out.println("Banana".compareTo("Apple")); // positive
3 System.out.println("Hello".compareTo("Hello")); // 0
4
5 // Case-insensitive version
6 System.out.println("hello".compareToIgnoreCase("HELLO")); // 0
```

JAVA

`compareTo()` is primarily used with `Arrays.sort()` and `Collections.sort()` — it defines the natural ordering of Strings.

QUICK REFERENCE — WHICH COMPARISON TO USE

- › Check if two strings have the same content → `a.equals(b)`
- › Same content, ignore case → `a.equalsIgnoreCase(b)`
- › Determine alphabetical order → `a.compareTo(b)`
- › Comparing with a potentially null variable → `"constant".equals(variable)`
- › Never `==` for content comparison — period

indexOf() — Find First Occurrence

```
1 String s = "Hello, World! Hello!";
2
3 System.out.println(s.indexOf("Hello"));    // 0
4 System.out.println(s.indexOf("World"));    // 7
5 System.out.println(s.indexOf("xyz"));      // -1 (not found)
6 System.out.println(s.indexOf("Hello", 5)); // 14 (search from index 5)
7 System.out.println(s.indexOf('o'));        // 4 (single char version)
8 System.out.println(s.indexOf('o', 5));     // 8
```

JAVA

lastIndexOf() — Find Last Occurrence

```
1 String s = "Hello, World! Hello!";
2 System.out.println(s.lastIndexOf("Hello")); // 14
3 System.out.println(s.lastIndexOf('o'));     // 16
4 System.out.println(s.lastIndexOf("Hello", 10)); // 0 (search backwards from index 10)
```

JAVA

contains() — Does It Contain a Substring?

```
1 String s = "Hello, World!";
2 System.out.println(s.contains("World")); // true
3 System.out.println(s.contains("world")); // false (case-sensitive)
4 System.out.println(s.contains("xyz"));   // false
```

JAVA

startsWith() and endsWith()

```
1 String s = "HelloWorld.java";
2
3 System.out.println(s.startsWith("Hello")); // true
4 System.out.println(s.startsWith("World")); // false
5 System.out.println(s.startsWith("World", 5)); // true (check from index 5)
6
7 System.out.println(s.endsWith(".java")); // true
8 System.out.println(s.endsWith(".cpp")); // false
```

JAVA

Remember: Strings are immutable. Every method below returns a **new String**. The original is never modified.

substring() — Extract a Portion

```
1 String s = "Hello, World!";
2
3 System.out.println(s.substring(7));    // "World!" (from index 7 to end)
4 System.out.println(s.substring(0, 5)); // "Hello" (from 0, up to but NOT including 5)
5 System.out.println(s.substring(7, 12)); // "World" (indices 7,8,9,10,11)
```

JAVA

MISTAKE

The second parameter is **exclusive** (not inclusive). `substring(0, 5)` gives characters at indices 0, 1, 2, 3, 4 — five characters total, not six.

concat() — Concatenate

```
1 String a = "Hello";
2 String b = a.concat(" World");
3 System.out.println(b);    // "Hello World"
4 System.out.println(a);    // "Hello" — original unchanged
5
6 // Same as using +
7 String c = "Hello" + " World"; // "Hello World"
```

JAVA

replace() and replaceAll()

```
1 String s = "Hello, World! Hello!";
2
3 // Replace ALL occurrences of a char or sequence
4 System.out.println(s.replace("Hello", "Hi")); // "Hi, World! Hi!"
5 System.out.println(s.replace('l', 'L'));      // "HeLLo, WorLd! HeLLo!"
6
7 // replaceAll uses REGEX
8 System.out.println(s.replaceAll("[aeiou]", "_")); // "H_ll_, W_rld! H_ll_!"
9 System.out.println(s.replaceAll("\\d+", "NUM")); // "Hello, World! Hello!" (no digits to replace)
10
11 // replaceFirst — only the first match
12 System.out.println(s.replaceFirst("Hello", "Hi")); // "Hi, World! Hello!"
```

JAVA

WARNING

`replace()` treats the first argument as a **literal string**. `replaceAll()` treats it as a **regex pattern**. If your replacement string contains regex special characters like `.`, `$`, `[`, use `replace()` — not `replaceAll()`.

trim() and strip() — Remove Whitespace

```

1 String s = "    Hello, World!    ";
2
3 System.out.println(s.trim());    // "Hello, World!" (removes ASCII whitespace only)
4 System.out.println(s.strip());    // "Hello, World!" (removes ALL Unicode whitespace – Java 11+)
5
6 // stripLeading / stripTrailing – one side only
7 System.out.println(s.stripLeading());    // "Hello, World!    "
8 System.out.println(s.stripTrailing());    // "    Hello, World!"

```

toUpperCase() and toLowerCase()

```

1 String s = "Hello, World!";
2 System.out.println(s.toUpperCase());    // "HELLO, WORLD!"
3 System.out.println(s.toLowerCase());    // "hello, world!"
4
5 // Original is unchanged – these return new strings
6 System.out.println(s);    // "Hello, World!"

```

repeat() — Repeat a String (Java 11+)

```

1 String dash = "-".repeat(10);
2 System.out.println(dash);    // "-----"
3
4 String row = "* ".repeat(5);
5 System.out.println(row);    // "* * * * *"

```

indent() — Add or Remove Indentation (Java 12+)

```

1 String s = "Hello\nWorld";
2 System.out.println(s.indent(4));
3 // "    Hello\n    World\n"
4
5 System.out.println(s.indent(-2));
6 // "Hello\nWorld\n" (if already unindented, minimum is 0)

```


split() — Break String into Array

JAVA

```
1 String csv = "Ali,Reza,Sara,Mina";
2 String[] names = csv.split(",");
3 // ["Ali", "Reza", "Sara", "Mina"]
4
5 // Split with limit (max number of parts)
6 String log = "2024-01-15 error: file not found";
7 String[] parts = log.split(" ", 2);
8 // ["2024-01-15", "error: file not found"]
9
10 // Split by regex (e.g., one or more spaces)
11 String data = "Hello World Java";
12 String[] words = data.split("\\s+");
13 // ["Hello", "World", "Java"]
```

WARNING

`split()` takes a **regex**, not a plain string. The dot `.` is a regex wildcard. To split by an actual dot, escape it:

`"file.txt".split("\\.")` → `["file", "txt"]`

`"file.txt".split(".")` → `[""]` (wrong! `.` matches everything)

Common split Patterns

GOAL	REGEX	EXAMPLE
Split by comma	<code>","</code>	<code>"a,b,c".split(",")</code> → <code>["a","b","c"]</code>
Split by spaces	<code>"\\s+"</code>	<code>"a b c".split("\\s+")</code> → <code>["a","b","c"]</code>
Split by dot	<code>"\\."</code>	<code>"a.b.c".split("\\.")</code> → <code>["a","b","c"]</code>
Split by pipe	<code>"\\ "</code>	<code>"a b c".split("\\ ")</code> → <code>["a","b","c"]</code>
Split by digits	<code>"\\d+"</code>	<code>"a12b3c".split("\\d+")</code> → <code>["a","b","c"]</code>

String.join() — Combine Array into String

JAVA

```
1 String[] names = {"Ali", "Reza", "Sara", "Mina"};
2
3 String result = String.join(", ", names);
4 System.out.println(result); // "Ali, Reza, Sara, Mina"
5
6 String path = String.join("/", "home", "user", "docs");
7 System.out.println(path);   // "home/user/docs"
```

String.valueOf() — Convert Anything to String

```

1 String a = String.valueOf(42);           // "42"
2 String b = String.valueOf(3.14);         // "3.14"
3 String c = String.valueOf(true);        // "true"
4 String d = String.valueOf('X');         // "X"
5 String e = String.valueOf(new int[]{1, 2, 3}); // "[I@..." (not useful for arrays!)
6
7 // For arrays, use Arrays.toString():
8 String f = Arrays.toString(new int[]{1, 2, 3}); // "[1, 2, 3]"

```

Parsing Strings Back to Numbers

```

1 int num = Integer.parseInt("42");
2 double dbl = Double.parseDouble("3.14");
3 long lng = Long.parseLong("9999999999");
4
5 // With validation
6 String input = "abc";
7 try {
8     int n = Integer.parseInt(input);
9 } catch (NumberFormatException e) {
10     System.out.println("Not a valid number!");
11 }

```

MISTAKE

`String.valueOf()` on an array does **not** give you a readable string — it prints the type and memory address. Use `Arrays.toString(arr)` or `Arrays.deepToString(arr)` instead (covered in Chapter 1).

new String(char[]) — char Array to String

```

1 char[] chars = {'H', 'e', 'l', 'l', 'o'};
2 String s = new String(chars);           // "Hello"
3 String s2 = new String(chars, 1, 3);    // "ell" (from index 1, length 3)

```


String.format() — Create a Formatted String

Same syntax as `printf`, but returns a new String instead of printing:

```
1 String name = "Ali";
2 int age = 20;
3 double gpa = 3.85;
4
5 String msg = String.format("Name: %s, Age: %d, GPA: %.2f", name, age, gpa);
6 System.out.println(msg);
7 // Name: Ali, Age: 20, GPA: 3.85
8
9 // Can also use the instance method (Java 15+)
10 String msg2 = "Name: %s, Age: %d".formatted(name, age);
11 System.out.println(msg2);
12 // Name: Ali, Age: 20
```

JAVA

Useful Format Specifiers for Strings

SPECIFIER	PURPOSE	EXAMPLE	OUTPUT
<code>%s</code>	String	<code>"%s".format("Hi")</code>	Hi
<code>%10s</code>	Right-pad to width 10	<code>"%10s".format("Hi")</code>	" Hi"
<code>%-10s</code>	Left-pad to width 10	<code>"%-10s".format("Hi")</code>	"Hi "
<code>%d</code>	Integer	<code>"%d".format(42)</code>	42
<code>%05d</code>	Zero-pad to width 5	<code>"%05d".format(42)</code>	00042
<code>%,d</code>	Comma-separated	<code>"%,d".format(1000000)</code>	1,000,000
<code>%.2f</code>	2 decimal places	<code>"%.2f".format(3.14159)</code>	3.14
<code>%n</code>	Platform newline	<code>"Line1%nLine2"</code>	Line1\nLine2
<code>%%</code>	Literal <code>%</code>	<code>"100%".format()</code>	100%

Practical Examples

```
1 // Table-like formatting
2 String header = String.format("%-15s %5s %8s", "Name", "Age", "GPA");
3 String row    = String.format("%-15s %5d %8.2f", "Ali Rezaei", 20, 3.85);
4 System.out.println(header);
5 System.out.println(row);
```

JAVA

Name Age GPA Ali Rezaei 20 3.85

OUTPUT

Why StringBuilder Exists

Since Strings are immutable, each `+` concatenation creates a **new object**. In a loop, this creates many throwaway objects and is very slow:

```
1  // SLOW – creates a new String object on every iteration
2  String result = "";
3  for (int i = 0; i < 10000; i++) {
4      result += i;  // each += creates a brand new String
5  }
6
7  // FAST – modifies a single buffer in-place
8  StringBuilder sb = new StringBuilder();
9  for (int i = 0; i < 10000; i++) {
10     sb.append(i); // modifies the buffer, no new objects
11 }
12 String result = sb.toString();
```

JAVA

TIP

Use `+` for simple one-liners — the compiler optimizes those. Use `StringBuilder` inside **loops** or when building a string from many pieces.

StringBuilder Methods

```

1  StringBuilder sb = new StringBuilder("Hello");
2
3  // append – add to the end
4  sb.append(", ");
5  sb.append("World");
6  sb.append("! ");
7  sb.append(2024);
8  System.out.println(sb); // "Hello, World! 2024"
9
10 // insert – add at a specific position
11 sb.insert(5, " JAVA");
12 System.out.println(sb); // "Hello JAVA, World! 2024"
13
14 // replace – replace a range [start, end)
15 sb.replace(6, 11, "Beautiful");
16 System.out.println(sb); // "Hello Beautiful, World! 2024"
17
18 // delete – remove a range [start, end)
19 sb.delete(5, 16);
20 System.out.println(sb); // "Hello, World! 2024"
21
22 // deleteCharAt – remove single character
23 sb.deleteCharAt(sb.length() - 1);
24 System.out.println(sb); // "Hello, World! 202"
25
26 // reverse
27 sb.reverse();
28 System.out.println(sb); // "202 !dlroW ,olleH"
29
30 // toString – convert back to String
31 String finalStr = sb.toString();

```

Chaining Methods

Most `StringBuilder` methods return `this`, so you can chain them:

```

1  String result = new StringBuilder()
2      .append("Hello")
3      .append(", ")
4      .append("World")
5      .append("!")
6      .toString();
7  System.out.println(result); // "Hello, World!"

```

Other Useful Methods

METHOD	RETURNS	DESCRIPTION
<code>length()</code>	<code>int</code>	Current length of the buffer
<code>capacity()</code>	<code>int</code>	Current buffer capacity (≥ length)
<code>charAt(i)</code>	<code>char</code>	Character at index <code>i</code>
<code>indexOf(str)</code>	<code>int</code>	First occurrence of <code>str</code>
<code>substring(start, end)</code>	<code>String</code>	Portion of the buffer
<code>setCharAt(i, ch)</code>	<code>void</code>	Replace character at index <code>i</code>
<code>toString()</code>	<code>String</code>	Convert to immutable String

StringBuilder vs StringBuffer

FEATURE	STRINGBUILDER	STRINGBUFFER
Thread-safe	No	Yes (synchronized methods)
Speed	Faster	Slower (synchronization overhead)
When to use	Single-threaded code (95% of cases)	Shared across multiple threads
Available since	Java 1.5	Java 1.0

QUICK REFERENCE — WHEN TO USE WHAT

- › One-line concatenation → `String s = "a" + "b" + "c";` (compiler optimizes this)
- › Building a string in a loop → `StringBuilder`
- › Modifying a string frequently → `StringBuilder`
- › Multi-threaded string building → `StringBuffer`
- › Final result needed as String → always call `.toString()` at the end

1. Using == Instead of equals()

```
1 Scanner sc = new Scanner(System.in);
2 String input = sc.nextLine();
3
4 // WRONG - almost always false for runtime strings
5 if (input == "yes") { }
6
7 // CORRECT
8 if (input.equals("yes")) { }
9
10 // SAFEST (null-safe)
11 if ("yes".equals(input)) { }
```

JAVA

2. Thinking Strings Are Mutable

```
1 String s = "Hello";
2 s.toUpperCase();
3 System.out.println(s); // "Hello" - NOT "HELLO"!
4
5 // You must assign the result:
6 s = s.toUpperCase();
7 System.out.println(s); // "HELLO"
```

JAVA

3. Forgetting split() Uses Regex

```
1 String filename = "report.final.txt";
2
3 // WRONG - "." matches any character
4 String[] parts = filename.split("."); // returns [""] - empty array essentially
5
6 // CORRECT - escape the dot
7 String[] parts = filename.split("\\."); // ["report", "final", "txt"]
```

JAVA

4. NullPointerException on equals()

```
1 String s = null;
2
3 // CRASH - NullPointerException
4 // if (s.equals("Hello")) { }
5
6 // SAFE - literal is never null
7 if ("Hello".equals(s)) { } // false, no crash
```

JAVA

5. String Concatenation in Loops

```

1  // SLOW - O(n^2) due to immutable string creation
2  String result = "";
3  for (String word : words) {
4      result += word + " ";
5  }
6
7  // FAST - O(n) with StringBuilder
8  StringBuilder sb = new StringBuilder();
9  for (String word : words) {
10     sb.append(word).append(" ");
11 }
12 String result = sb.toString();

```

6. substring() Off-by-One

```

1  String s = "Hello World";
2
3  // Want "World" (indices 6 through 10)
4  // WRONG - index 11 is out of bounds
5  // s.substring(6, 11)
6
7  // CORRECT - end is EXCLUSIVE
8  String word = s.substring(6, 11); // "World"

```

7. Comparing with equals() After toUpperCase()

```

1  String input = "hello";
2
3  // Works but creates a new string object
4  if (input.toUpperCase().equals("HELLO")) { }
5
6  // More efficient - no new object needed
7  if (input.equalsIgnoreCase("HELLO")) { }

```

8. Not Handling Empty Strings in split()

```

1  String s = ",a,,b,c,";
2
3  // split by default drops trailing empty strings
4  String[] parts1 = s.split(",");
5  // [ "", "a", "", "b", "c" ]
6
7  // To keep ALL empty strings, use negative limit
8  String[] parts2 = s.split(",", -1);
9  // [ "", "a", "", "b", "c", "" ]

```

QUICK REFERENCE — STRING DEBUGGING CHECKLIST

- › Comparison fails → switch `=` to `.equals()`, or use `"literal".equals(var)` for null safety
- › String not changing → you forgot to assign the return value (strings are immutable!)
- › `split()` gives wrong result → the delimiter is regex — escape special chars: `\\.`, `\\|`, `\\\\`
- › `NullPointerException` → check for null before calling methods, or put literal first
- › Performance issue in loop → replace `+` with `StringBuilder`
- › `StringIndexOutOfBoundsException` → check `length()` before `charAt()` or `substring()`
- › Print the string with `System.out.println("[ " + s + " ]")` to see hidden whitespace

String Methods Quick-Lookup Table

CATEGORY	METHODS
Length & chars	<code>length()</code> , <code>charAt()</code> , <code>toCharArray()</code> , <code>isEmpty()</code> , <code>isBlank()</code>
Comparison	<code>equals()</code> , <code>equalsIgnoreCase()</code> , <code>compareTo()</code> , <code>contentEquals()</code>
Search	<code>indexOf()</code> , <code>lastIndexOf()</code> , <code>contains()</code> , <code>startsWith()</code> , <code>endsWith()</code>
Extract	<code>substring()</code> , <code>subsequence()</code>
Modify	<code>concat()</code> , <code>replace()</code> , <code>replaceAll()</code> , <code>replaceFirst()</code>
Whitespace	<code>trim()</code> , <code>strip()</code> , <code>stripLeading()</code> , <code>stripTrailing()</code>
Case	<code>toUpperCase()</code> , <code>toLowerCase()</code>
Split & join	<code>split()</code> , <code>String.join()</code>
Convert	<code>String.valueOf()</code> , <code>Integer.parseInt()</code> , <code>Double.parseDouble()</code>
Format	<code>String.format()</code> , <code>formatted()</code>
Other	<code>repeat()</code> , <code>indent()</code> , <code>intern()</code> , <code>matches()</code> , <code>lines()</code>

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Programiz — Java Strings**

String creation, methods, immutability, StringBuilder guide

<https://www.programiz.com/java-programming/string>

[2] **GeeksforGeeks — String in Java**

String Pool, string methods reference, StringBuilder vs StringBuffer comparison

<https://www.geeksforgeeks.org/string-class-in-java/>

[3] **Oracle — Java Documentation — `java.lang.String`**

Official API reference for all String class methods

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

[4] **Baeldung — Java String**

Guides on String immutability, StringBuilder best practices, split pitfalls

<https://www.baeldung.com/java-string>

[5] **W3Schools — Java Strings**

Quick reference for string methods and syntax

https://www.w3schools.com/java/java_strings.asp

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com